Seeing by Coding: Active Perception Lifts Document Agents Across Model Families

Barış Deniz Sağlam
COGS 560 Term Project
e155841@metu.edu.tr

Abstract

Document visual question answering on long, multi-page, high-resolution documents resists both small agents and frontier-scale frozen models: on the ICDAR 2026 DocVQA validation set, bare setups score 17–22 ANLS and frontier models plateau near 33–38. We show that the dominant lever is not model scale but *active perception* — giving a code-capable reasoner a persistent Python REPL and a single on-demand call to a frozen vision–language model, so it can crop, zoom, and compute over document regions rather than read them whole. Holding both models fixed and varying only the scaffold spans roughly 25 ANLS points; a REPL active-perception scaffold on a 27B VLM reaches ∼39 ANLS, matching or exceeding much larger frozen models and far surpassing tool-based ReAct. The advantage concentrates on visually dense pages, collapses when vision is replaced by OCR, and generalizes across model families given a capable-enough base. Because the append-only member of this scaffold family preserves the growing-prefix trajectory that weight-level training assumes, we use it to ask, in a preliminary study, whether fine-tuning can lift a $\leq$8B agent further; at matched sample size the supervised stage ties the untrained base, locating the lever in the on-policy methods we leave to future work.

1 Introduction

The documents in DocVQA-2026 are long, multi-page, high-resolution, and largely visual. A single question can range over dozens of pages of figures, plots, schematics, maps, and infographics, and the answer-bearing content is frequently non-textual — a value in a chart cell, a label on an engineering drawing, a region of a poster. Because much of the signal never resolves to text, OCR-and-text pipelines cannot reach it. Answering is, first, a problem of *finding*: locating the right page and the right region before any reading can begin, then often combining or computing over what is found.

Scaling the model does not by itself crack this. On the DocVQA-2026 validation split, bare setups score low — a raw multi-image VLM call, a multimodal agent reading pages in its own context, and a competition-kit prompt with no scaffold all land near 17–22 ANLS. Frontier-scale frozen models do better but only plateau: the official ICDAR-2026 baselines place Gemini 3 Pro at 37.5 and Gemini 3 Flash at 33.75 ANLS on the same validation split, matching or modestly exceeding the bare agents but low for their size on documents this large. Both ends of the scale axis fall short of what the task affords, which points away from model capacity as the binding constraint.

What moves accuracy is *active perception*. Give a code-capable reasoner a persistent Python REPL and a single visual primitive — an on-demand call to a frozen VLM perception tool against an arbitrary image region — and the reasoner can direct perception rather than consume it whole: crop to the evidence, zoom for acuity, and do in code the coordinate and numeric arithmetic the VLM cannot. Holding both the reasoner and the frozen VLM fixed and varying only this scaffold spans roughly 25 ANLS points. The REPL active-perception scaffolds reach ∼39 ANLS — far above tool-based ReAct, which has the same VLM but no REPL (∼25), and above no-REPL VLM agents (∼20) — and match or exceed the much larger frontier frozen models above. The result is demonstrated chiefly at a Qwen3.5-27B backbone, but it is not specific to one model: it holds across model families and is gated by base capability rather than size.

A second thread follows from the structure of the winning scaffold. Active perception is a *family* of harnesses, and its append-only member preserves a growing-prefix trajectory — the structure that weight-level training assumes — at no mea-

sured accuracy cost, making it the principled target for fine-tuning. In a preliminary study, we ask whether training a small reasoner (Qwen3.5-4B) inside that member can lift a ≤8B agent further toward the gain.[1]

We make the following contributions.[2]

1. **Active perception is the dominant lever on document-agent accuracy.** A controlled comparison on DocVQA-2026 val (binary ANLS at threshold 0.9, n=8, frozen Qwen3.5-27B VLM) shows REPL active-perception scaffolds (∼39 ANLS) far surpassing tool-based ReAct (∼25) and no-REPL VLM agents (∼20), and matching or exceeding the official ICDAR-2026 frontier baselines on the same validation split (Gemini 3 Pro 37.5, Gemini 3 Flash 33.75 — low for their size on documents this large). The ordering is robust across two model families (Qwen3.5, Gemma-4) and four reasoner sizes (§3): the lever is the scaffold, not the model.

2. **Why and when active perception works.** The code REPL and visual perception are each essential — dropping either collapses the score — and the advantage concentrates on *visually dense* pages, where cropping recovers detail a whole-page read misses (the lead over ReAct is largest on the detail-dense categories: engineering drawings, business reports, maps; §3.5); OCR-free visual perception is decisive (swapping vision for OCR text costs −25.5 points, with engineering drawings and maps falling to zero). The mechanism is that the REPL converts reasoning capability into targeted perception — which is also why the advantage is *capability-gated*, not scale-gated: a modern Qwen3.5-4B clears the bar, while a same-size Gemma-4-E4B (no lift) and an older, larger Qwen3-8B (where the REPL actively hurts a weak coder) are clean negative controls.

3. **A preliminary ≤8B training study.** We build a rejection-sampling fine-tuning pipeline and a training-data pool for the trainable scaffold and run a preliminary LoRA study on Qwen3.5-4B; reinforcement learning and on-policy distillation are motivated but left to future work. At matched sample size the best supervised configuration ties the untrained base rather than beating it — a null that is consistent across the study and points to on-policy training, not off-policy imitation, as the lever.

The remainder proceeds from background and task (§2) to the active-perception evaluation and its mechanism (§3), the argument for the append-only scaffold as a trainable target (§4), the preliminary small-agent training study (§5), related work (§6), and conclusions (§7).

2 Background

2.1 The DocVQA 2026 benchmark

Document Visual Question Answering asks a system to answer natural-language questions about a document presented as page images rather than as parsed text (Mathew et al., 2020). The ICDAR 2026 DocVQA benchmark targets the hard end of this task. Its documents are long, multi-page, and high-resolution, and they span eight content categories — including dense business reports, scientific posters, maps, and engineering drawings — so that answering a single question often requires locating the relevant page and region before reading it. The benchmark provides no training split: the validation set holds 25 documents and 80 questions, and the test set holds 48 documents and 160 questions. Models must therefore be developed without in-distribution supervised data, which places weight on transfer from related corpora and on the design of the inference-time agent.

Answers are scored with the Average Normalized Levenshtein Similarity (ANLS), introduced for scene-text VQA to give partial credit for near-miss strings while penalizing substantive errors (Biten et al., 2019). ANLS is one minus the normalized Levenshtein edit distance between a prediction and a reference, averaged over questions. The 2026 protocol applies it in a binary form: a

[1] The original proposal envisioned a fuller progression — supervised fine-tuning, then reinforcement learning, then on-policy distillation. We report the supervised stage and leave the on-policy methods to future work. Agentic reinforcement learning in this setting is bottlenecked by on-policy rollout generation — each update needs many long, multi-turn trajectories, and every perception step queries a slow frozen vision–language model — and the hybrid-attention backbone added integration overhead newer to the open training stack; within the project's time budget, the controlled harness evaluation (§3) was the more valuable investment.

[2] Code is available at `https://github.com/bdsaglam/docvqa` (evaluation) and `https://github.com/bdsaglam/docvqa-verl` (fine-tuning).

per-question similarity at or above a threshold of 0.9 counts as a correct answer and anything below it as wrong, so the reported score is effectively a thresholded accuracy. This binary ANLS@0.9 governs every number in this report; the evaluation protocol is given in full in §3.1.

Two properties of the documents shape the difficulty. First, much of the answer-bearing content is visual rather than textual — figures, plots, schematics, and map labels — where optical character recognition is unreliable or simply inapplicable, so a text-only pipeline cannot reach the relevant evidence. Second, many questions are multi-hop or arithmetic, requiring evidence to be gathered from several locations and then combined or computed rather than read off a single span. Both properties favor a system that can perceive document regions selectively and reason over what it perceives, rather than one that reads a fixed transcription of the page.

2.2 Code-REPL agents for document VQA

The agents studied here pair a frozen vision–language model (VLM) for perception with a code-capable language model (LM) as the reasoner. The reasoner does not view the document pixels directly; instead it writes code that issues perception requests against the VLM — cropping, zooming, and querying specific page regions — receives the returned observations into its context, and continues reasoning until it commits a final answer. Perception is thus *active*: rather than consuming a single fixed rendering of the document, the reasoner decides what to look at next based on what it has already seen, iteratively, and on demand. This active visual perception is what lets the agent reach content OCR cannot and chain evidence across pages. The VLM stays a frozen external endpoint throughout; what varies between agents is the reasoning LM and the harness that connects it to perception.

These agents form a family. A harness exposes the same perception interface — most importantly a code REPL in which the reasoner programs its visual queries — but members differ in how they manage the growing context and which tools they surface; we refer to the members built around a code REPL and `batch_look`-style perception as the REPL active-perception scaffold family. A prior competition entry, submitted under the name Perceive–Reason–Code, instantiates one member: a 27B-parameter reasoner driving such a harness over a frozen VLM, placed in the 8B–35B parameter tier of the leaderboard. That entry is one instance of the family, not a self-evidently best design, and it marks the prior-work boundary for our study.

What is reused from that prior work is the scaffold — the active-perception harness and its tool surface. What is new here is twofold. First, a controlled *harness evaluation*: rather than taking the active-perception design as given, we measure it against alternative agent designs to isolate which structural choices drive accuracy (§3). Second, *weight-level training* of a small ($\leq$8B) reasoning LM inside the trainable member of the family, asking whether fine-tuning can lift a small agent further rather than deploying a larger frozen reasoner zero-shot (§5). The scaffold is inherited; both the evaluation of it and the training within it are our contributions.

3 Active Perception Lifts Document Agents

A frozen vision-language model and a code-capable reasoner can be wired into a DocVQA agent in many ways, and the wiring — not the choice of either model — is what moves accuracy. Holding both models fixed and varying only the scaffold spans roughly 25 ANLS points on the DocVQA-2026 validation set. We establish the challenge (the task resists both small and frontier-scale models), state the active-perception hypothesis, prove it under a controlled comparison, isolate the mechanism, and show it generalizes across model families.

3.1 Evaluation setup

The metric is binary ANLS at threshold 0.9, the official DocVQA-2026 scoring rule: a prediction earns credit only when its normalized Levenshtein similarity to the gold answer reaches 0.9, otherwise zero. All harness numbers below are reported as mean $\pm$ std over n=8 independent trials on the DocVQA-2026 validation split (25 documents, 80 questions), micro-averaged per question, with a frozen Qwen3.5-27B (served with vLLM (Kwon et al., 2023)) as the VLM perception endpoint throughout. The reasoner's native thinking mode is disabled.

Disabling native thinking does not remove reasoning — it *relocates* it. These models emit their reasoning by default in a dedicated thinking chan-

nel; we turn that channel off, and because the harness parses only the fenced Python code, the model instead writes its reasoning as free text interleaved with its code actions. Reasoning is thus present every turn, inline rather than in a separate channel — and the DSPy solvers (Khattab et al., 2023) make it explicit, declaring a reasoning field in each turn's signature. Empirically the two settings are on par: disabling native thinking costs no measurable accuracy, while native thinking was hang-prone at the 27B scale and slower. No-think is therefore the stable, efficient configuration, not an answer-without-reasoning handicap.

3.2 The challenge: bare and frontier setups both fall short

DocVQA-2026 documents are long, multi-page, high-resolution, and largely visual — figures, plots, schematics, maps — so the answer-bearing content is often non-textual and a single question usually requires locating the right page and region before reading it. The task resists both ends of the scale axis. In a bare setup, no-scaffold prompting and raw multi-image VLM calls score low: a single raw multi-image call reaches 20.47 ANLS, a single multimodal agent reading pages in its own context 22.34, and the official competition baseline 17.81. At the other end, frontier-scale frozen models only plateau — the official ICDAR-2026 baselines place Gemini 3 Pro at 37.50 and Gemini 3 Flash at 33.75 on the same validation split, low for their size on documents this large. Scaling the model alone is not the answer.

3.3 Harness design space and the active-perception hypothesis

A document agent's design space spans a taxonomy of scaffolds, all sharing the same frozen Qwen3.5-27B VLM:

- **Raw multi-image VLM** — one VLM call over all page images, no agent, using the *same* minimal prompt as the active-perception scaffolds, so the only difference from a scaffold is the scaffold itself (the controlled no-scaffold baseline).

- **Multimodal REPL agent** — a single multimodal agent that pulls document pages into its own context with `display()` and reasons over them directly, with no VLM-tool call.

- **Official baseline** — the published DocVQA-2026 competition baseline: one VLM call with the kit's *own* prompt (an external reference point). It is the same raw-VLM mechanism as the row above and differs only in the prompt.

- **ReAct** — a thought → tool → observation loop with the same VLM perception tools but **no Python REPL**. Its only perception actuators are whole-page VLM queries at fixed page granularity; it cannot crop, zoom, or do coordinate arithmetic.

- **the REPL active-perception family** — scaffolds whose action is Python executed in a persistent REPL, sharing a tool surface, prompt body, and the perception primitive `batch_look`, an on-demand call to the frozen VLM perception tool against cropped or zoomed image regions. The family has two members. **RLM** (Recursive Language Models (Zhang et al., 2025)) holds state in a hidden interpreter namespace and compacts the rendered context across turns. **CodeAct** (Wang et al., 2024) is its append-only twin — identical tools and `batch_look` perception call, but a strictly append-only chat transcript in which each turn extends the previous one and no earlier token is rewritten, the prefix-preserving property §4 requires of a fine-tuning target. Neither reads OCR text; perception is purely visual.

- **OCR-only control** — the RLM scaffold with visual perception swapped for OCR text plus lexical search (OCR-only RLM), isolating the contribution of the visual modality.[3]

The hypothesis follows: what governs accuracy is the **REPL active-perception loop** — code that issues on-demand visual queries (crop, zoom, coordinate arithmetic) against a frozen VLM and continues reasoning over what comes back — not model scale and not a fixed-granularity tool loop. Figure 1 contrasts the active-perception scaffold with the REPL-less ReAct loop.

[3] OCR text is pre-extracted per page with Docling (figures and pictures described by a Granite-Vision model) and indexed for BM25 lexical search; the agent reads this text in place of querying the VLM.

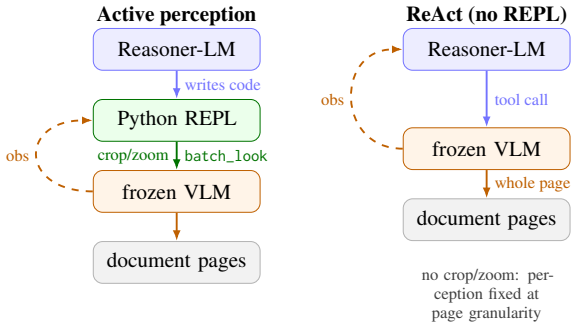

Figure 1: The active-perception scaffold (left) inserts a Python REPL between reasoner and VLM, so the reasoner programs its perception — cropping and zooming to image regions via `batch_look`. ReAct (right) calls the same frozen VLM but without a REPL, so its only actuator is a whole-page query at fixed granularity. The REPL is the sole structural difference, and it is what converts reasoning into targeted perception (§3.5).

3.4 Result: active perception dominates

Table 1 compares the distinct harness designs under the common protocol, alongside two external frontier anchors; we defer ablations within a single design to §3.5. The designs fall into two tiers separated by a gap several times the within-tier standard deviation.

The active-perception designs (RLM, CodeAct) reach ∼39 ANLS; every design that drops the REPL or the perception tool falls to 18–25. RLM, the reference active-perception result, reaches 39.38 and the append-only CodeAct ties it at 39.53 (within trial-to-trial variance), so the win belongs to the active-perception *family* rather than a single harness — a tie §4 builds on. The family also matches or exceeds the official ICDAR-2026 frontier baselines on the same validation split (Gemini 3 Pro 37.50, Gemini 3 Flash 33.75): the advantage is not scale, since an active-perception scaffold on a 27B VLM reaches the accuracy of proprietary models many times larger.

3.5 Mechanism: which components matter

The mechanism is isolated two ways: by comparing RLM against the harness designs in Table 1 that drop one of its components, and by ablating one lever at a time within the RLM solver itself (Table 2). We keep the two apart deliberately — Table 1 contrasts different designs; Table 2's ablations all act on RLM.

The REPL and the VLM perception tool are each necessary. Removing either collapses the score. Stripping the REPL but keeping the perception tool (ReAct) drops the score by 14.2 relative to RLM; stripping the perception tool but keeping the REPL — the Multimodal REPL agent, which loads pages into its own context instead — drops it by 17.0; the no-agent baselines that drop both (Raw multi-image VLM 20.47, Official baseline 17.81) sit lower still. Neither half alone suffices.

Within RLM, cropping and the visual modality are necessary; extra channels are not. Table 2 ablates one lever of the RLM solver at a time. Two levers carry the result. Restricting `batch_look` to whole-page reads (removing crop/zoom) costs -2.5 overall but hits the detail-dense categories hardest — engineering drawings and science posters, where zoom matters most; and replacing visual perception with OCR text collapses accuracy by 25.5 points to 13.91. The other directions are null-to-harmful: adding OCR text atop vision (-1.6), adding a direct `display()` image channel (-3.9), and generalizing the perception call into an arbitrary-subtask delegation tool (-0.16) none helps — the lean, perception-only `batch_look` is already at the optimum, and the extra channels mostly add noise.

Both necessary levers act through **visual density**, not document length. At matched perception (a shared VLM, so the difference is purely harness), the RLM$-$ReAct gap is largest on visually dense categories — engineering drawings, business reports, maps, infographics — and smallest on text-linear ones — scientific papers and slides (Figure 2); with a strong VLM the advantage is flat to slightly negative on the longest documents, where a whole-page read suffices once the answer page is found. The OCR-only collapse is graded the same way: the OCR-only agent scores **0/10 in all eight trials** on engineering drawings and maps, where the page is non-textual, and is most survivable on text-dense slides and scientific papers. Vision does the work OCR cannot, exactly where the content is not text. (These per-category gaps are computed on cross-model matched-config runs; the canonical absolutes are the n=8 numbers in Tables 1–2.)

The REPL converts reasoning into perception. A factorial swap separates the harnesses by what they are bound on. Replacing a weak reasoner on the 27B VLM with a strong 27B reasoner on a weaker 9B VLM *gains* $+10.3$ for RLM and $+6.2$ for CodeAct (both reasoning-bound) but *loses* -3.05 for ReAct (perception-bound). The

Tier	Solver	Mechanism	ANLS (n=8)
active perception	RLM	REPL + VLM perception tool `batch_look`, OCR-free	**39.38 ± 1.49**
	CodeAct	append-only twin, same tools	**39.53 ± 2.83**
no active perception	ReAct	VLM tool, no REPL	25.16 ± 4.60
	Multimodal REPL agent	REPL, pages in own context, no VLM-tool call	22.34 ± 2.79
	Raw multi-image VLM	single VLM call, prompt matched to scaffolds	20.47 ± 1.63
	Official baseline	competition kit's own prompt, one VLM call	17.81 ± 1.86
external anchor	Gemini 3 Pro	—	37.50 (val)
	Gemini 3 Flash	—	33.75 (val)

Table 1: Harness-design comparison on DocVQA-2026 val (80Q), n=8, frozen Qwen3.5-27B VLM, no-think. The two tiers are separated by a gap several times the within-tier standard deviation. CodeAct is the corrected append-only loop (final, n=8).

RLM ablation	ANLS	Δ	what it changes
RLM (reference)	39.38	—	perception-only `batch_look`, crop/zoom, OCR-free
+ multi-modal sub-agent	39.22	−0.16	generalize perception to any subtask
+ OCR	37.81	−1.56	add OCR text + lexical search atop vision
− crop/zoom	36.88	−2.51	whole-page reads only
+ display channel	35.47	−3.91	add a direct `display()` image channel
OCR-only	13.91	−25.47	replace vision with OCR text

Table 2: Ablations of the RLM solver — one lever changed at a time, vs RLM (39.38), n=8 (ANLS@0.9, ±std omitted; all within 1.5–4.5).

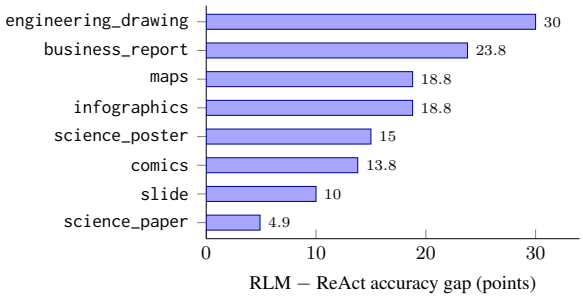

Figure 2: The active-perception advantage tracks **visual density**. At matched perception (a shared 27B VLM, so the difference is purely harness), the RLM−ReAct accuracy gap by category is largest on detail-dense pages where cropping recovers fine structure (engineering drawings, business-report figures, maps, infographics) and smallest on text-linear ones (slides, scientific papers). Per-category cells are noisy (10 questions each); the ranking, not the exact value, is the point.

REPL members write Python around `batch_look` — cropping to the evidence region, zooming, chaining successive crops, doing coordinate arithmetic — so a stronger reasoner produces better-targeted perception queries and extracts more even from a weaker VLM. ReAct has no such actuator: its perception ceiling is the VLM's whole-page acuity, which a stronger reasoner cannot direct. The complementary swap confirms a perception bottleneck for mid and small reasoners: holding the reasoner fixed and upgrading its homogeneous VLM to the 27B VLM lifts accuracy +7.9 at the 9B reasoner (Welch t=3.54, 95% CI [+3.4, +12.3]) and +8.6 at the 4B reasoner (t=4.96, 95% CI [+5.2, +12.0]). The crop/zoom loop is the mechanism that turns reasoning capability into perception quality; remove it and reasoning can no longer buy perception.

Effort is not accuracy. Iteration count correlates negatively with success: $\mathrm{corr}(\text{iterations}, \text{accuracy}) \approx -0.31$ over 200 doc-trials, with accuracy falling $62.8\% \rightarrow 43.2\% \rightarrow 31.6\%$ across the 0–8 / 8–14 / 14+ iteration bins. The active-perception harnesses spend roughly twice the iterations and wall time of ReAct, but extra turns mark a hard document, not a path to the answer; the lever is the quality of the perception loop, not its length.

A trajectory. Figure 3 shows the loop on a single question over a 181-page document, trimmed verbatim from one agent trajectory. The agent surveys the document programmatically, locates the target table through a table-of-contents pointer,

Reasoner (× 27B VLM)	RLM	ReAct	CodeAct
Qwen3-8B (older gen)	11.73	15.79	9.50†
Qwen3.5-4B	21.09	15.66	22.34
Qwen3.5-9B	24.54	21.01	24.26†
Qwen3.5-27B	39.38	25.16	39.53

Table 3: Reasoner scale at fixed perception — each reasoner paired with the 27B VLM. DocVQA-2026 val, n=8.

Model (homogeneous)	RLM	ReAct	CodeAct
Qwen3.5-4B	12.49	11.94	16.25
Qwen3.5-9B	16.67	14.97	19.35†
Qwen3.5-27B	39.38	25.16	39.53
Gemma-4-31B	32.50	18.44	29.25†
Gemma-4-E4B	7.34	6.09	7.66†

Table 4: Cross-family, homogeneous models — the reasoner is also its own VLM. DocVQA-2026 val, n=8.

distrusts the full-page VLM read and re-reads a tighter crop — which disagrees, exposing a VLM misread — adjudicates by reading the region in halves, and finally does the arithmetic the VLM cannot do, in Python. Every component of the mechanism appears in one trace, and the self-distrusting crop-verify is consequential rather than ceremonial: it catches a wrong number the whole-page read returned.

The same mechanism shows as a head-to-head contrast. On `science_poster_1_q1` — "the percentage score improvement from Baseline to TexTok in rFID-50k for ImageNet 512×512 with 128 tokens", answer `30.2%`, which requires reading two cells (`1.49`, `1.04`) from a small table embedded in a 4000×2000 poster and computing $((1.49 - 1.04)/1.49) \times 100$ — the RLM agent crops the table band, reads each operand with `batch_look`, and computes the result in Python, returning `30.2%` (correct) in six iterations. ReAct, with no REPL, reads the whole 4000×2000 poster at page granularity, cannot zoom to the cell, terminates in two iterations, and returns `0.48%` (wrong). The REPL is the difference between reading a cell and reading a page.

3.6 Generalization: cross-family and capability-gated

The advantage is not specific to one model. Two sweeps probe its reach without conflating perception with reasoning: one varies the reasoner while holding perception fixed at the 27B VLM (Table 3), the other compares whole homogeneous models, where the reasoner is also its own VLM, across families (Table 4).

With perception held fixed, RLM is at or above ReAct at every Qwen3.5 reasoner size (4B 21.09 vs 15.66, 9B 24.54 vs 21.01, 27B 39.38 vs 25.16), and the corrected CodeAct ties RLM at the two scales where it is complete — 22.34 vs 21.09 at 4B and 39.53 vs 39.38 at 27B. The older-generation Qwen3-8B is the informative exception: given the same strong 27B perception, it still scores higher under ReAct (15.79) than under RLM (11.73) or CodeAct (9.50). A weak coder cannot drive a code REPL, so handing it one hurts; with perception held constant, the binding constraint here is coding capability.

Within the Qwen3.5 family the homogeneous ladder rises only with scale: 4B and 9B sit low (RLM 12.49 and 16.67), where reasoner and perception are jointly weak, and only the 27B clears the bar — and the same 4B nearly doubles, to 21.09, once paired with the strong 27B VLM (Table 3), underscoring the perception bottleneck of §3.5. The harness ordering then reproduces on a second model family: Gemma-4-31B shows RLM 32.50 ≫ ReAct 18.44 (a +14pp gap), mirroring Qwen3.5-27B's 39.38 ≫ 25.16. The small homogeneous Gemma-4-E4B is the floor control — all three harnesses land at 6–8, within noise of the ∼6.25 official baseline, so at this capacity no scaffold pays off.

Together the sweeps fix the gate as base capability — reasoning, coding, and vision jointly — not raw parameter count. A modern Qwen3.5-4B clears it (Table 3); a same-size but weaker Gemma-4-E4B does not (Table 4); and an older Qwen3-8B fails even with strong perception, for lack of coding ability (Table 3). The advantage appears once, and only once, the base can drive the loop. († older CodeAct implementation, provisional pending rerun; the 4B and 27B CodeAct cells are the corrected append-only loop, final at n=8.)

4 From Harness to Trainable Scaffold

The §3 evaluation establishes a *family* of active-perception scaffolds, not a single winning harness: at the Qwen3.5-27B backbone the two strongest members, RLM and CodeAct, attain the highest accuracy and are statistically indistinguishable (39.4 and 39.5 ANLS, n=8), sharing a tool set, prompt body, and VLM-tool perception. The win is the family. The supervised study of §5 trains one

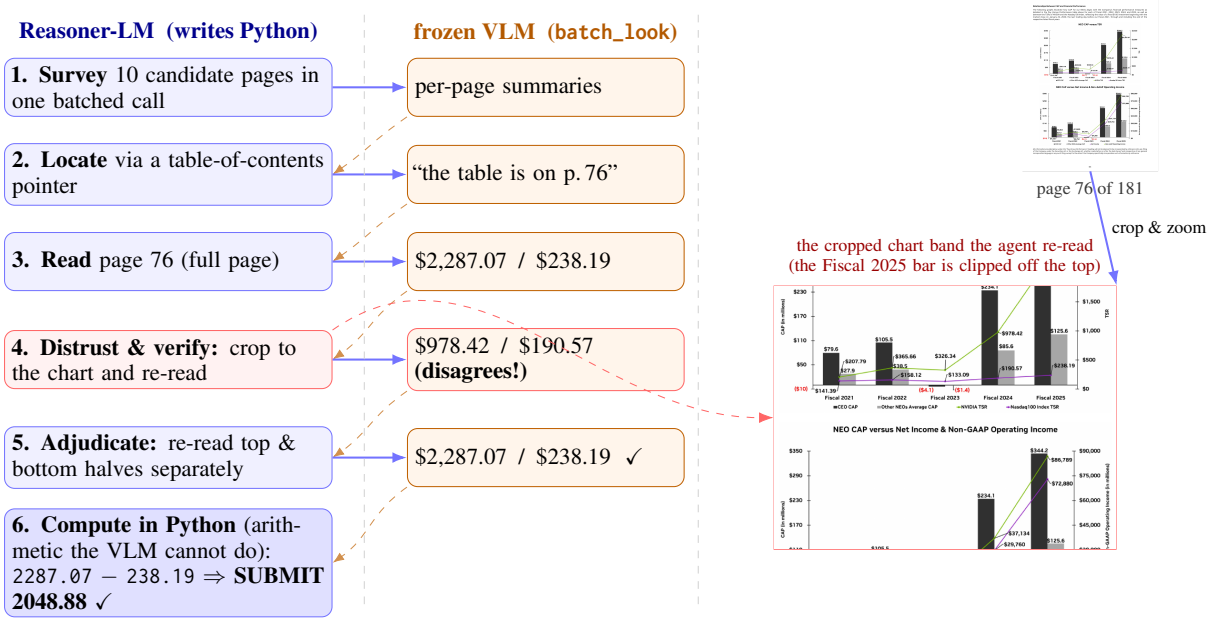

Figure 3: Active-perception loop on a 181-page business report (NVIDIA annual review), RLM, 16 iterations, **correct** (gold and predicted 2048.88). Question: *"In Fiscal 2025, by how many dollars does NVIDIA's TSR value exceed the Nasdaq-100 Index TSR value?"* The reasoner surveys the document, locates the table via a table-of-contents pointer, then **distrusts the full-page read and re-reads a tighter crop** (right) — which disagrees, exposing a perception error — adjudicates by reading the region in halves, and does the arithmetic in Python. Every component of the mechanism appears in one trace; the crop-verify catches a wrong number a single read would have submitted.

member, the append-only CodeAct; the on-policy methods left to future work — policy-gradient reinforcement learning and per-token distillation — single it out for a sharper reason. The two harnesses differ in how each turn's context is formed across turns, and that difference — not accuracy — is what determines whether those on-policy objectives are even well-defined. This section makes that argument: it is why the append-only member is the scaffold to train inside, and the analysis applies whether the training has happened yet or, as for the on-policy methods here, lies ahead.

4.1 The append-only prefix invariant

Policy-gradient multi-turn reinforcement learning (Group Relative Policy Optimization, GRPO; Proximal Policy Optimization, PPO; GSPO) and per-token distillation losses are built on an append-only token-prefix invariant: $\text{prompt}_{t+1} = \text{prompt}_t + \text{completion}_t + \text{observation}_t$. Under this invariant the whole rollout is one monotonically growing token sequence — the context at turn t is an exact prefix of the context at turn $t{+}1$ — so the trainer recovers every per-step log-probability from a single masked forward pass over the trajectory, scoring each action in exactly the context that generated it (Gallouédec and Rasul, 2025).

CodeAct preserves this invariant. It maintains an append-only chat transcript — a system message followed by alternating user and assistant turns — in which each turn only appends and no earlier token is rewritten. RLM does not. It surfaces a sidecar of metadata describing the variables currently live in the REPL, holds their full values off-context in the interpreter, and re-renders this managed view of state at each turn rather than appending to it. Because the visible context is rebuilt every step, earlier positions change from turn to turn and consecutive turns do not stand in a prefix relation.

Breaking the invariant breaks the policy gradient. Rewriting history makes the observation distribution policy-dependent and non-stationary, violating the assumptions of policy-gradient methods (Shao et al., 2025), and the rollout is no longer a single growing sequence, so per-step likelihoods must be recovered from separate forward passes over distinct contexts (Wu et al., 2025). Importance ratios and cross-turn credit assignment are then no longer defined over one coherent sequence. Methods that let an agent compress its own history during reinforcement learning report exactly

this and repair it with purpose-built machinery: FoldAct stabilizes summary actions with custom losses, and ReSum segments the trajectory at each compression and broadcasts a trajectory-level advantage across segments. An append-only transcript is the structure these objectives are built for; a context-manipulating one needs bespoke repair.

4.2 Prefix preservation is free

Selecting the append-only member concedes no accuracy where it is measured. With the corrected CodeAct loop, CodeAct ties RLM at both scales evaluated under matched perception — 39.5 against 39.4 at the 27B backbone, and 22.3 against 21.1 at the 4B reasoner that the training in §5 actually targets (n=8, both differences within trial-to-trial variance). The structural property that makes CodeAct trainable thus comes at no measured accuracy cost relative to RLM's managed, re-rendered context — including at the small scale where it matters for training. (On some bases RLM remains the more robust of the two, and the remaining cross-model CodeAct cells await the same corrected re-run.) The conclusion the bridge carries forward is the family-level one — train inside the append-only member of the winning family, paying nothing in accuracy for a trajectory that established training machinery accepts directly.

4.3 An accessible ceiling versus a higher one

The two scaffolds differ in how context length scales. RLM's managed view stays compact across arbitrarily many turns, decoupling the number of reasoning steps from the context window, whereas CodeAct's append-only transcript grows monotonically and can approach the window limit on long documents with many perception calls. In principle this gives RLM a higher ceiling on long-horizon tasks. That the advantage does not surface as higher accuracy is consistent with a specific account: exploiting a compact, re-rendered context requires a policy trained to operate on it, but contemporary models are post-trained almost entirely on append-only conversations, leaving a frozen model off-distribution in such a harness. Managed-context systems indeed tend to match or beat append-only baselines only after regime-specific training — an untrained context curator underperforms full-context prompting and overtakes it only once trained with reinforcement learning (Li et al., 2026), and stable context folding requires purpose-built losses (Shao et al., 2025).

This account is a hypothesis, not a demonstrated result, and its magnitude should not be overstated: frozen models sometimes benefit from compaction with no retraining (Wu et al., 2025), and that benefit shrinks as model capability grows. The conservative conclusion is that an append-only scaffold delivers its full potential under established training, whereas a context-managing scaffold may have a higher but currently less accessible ceiling, reachable only through new and not-yet-established methods. We adopt the prefix-preserving scaffold, for which mature training machinery applies directly; training a policy natively for a context-managing harness is left to future work. The cost of this choice is the append-only transcript's monotonic growth, which on long documents can press against the context window — a limitation for the longest documents that this preliminary study does not attempt to quantify.

5 Training a Small Agent

The §4 analysis fixes the target of weight-level training: the append-only CodeAct scaffold, whose growing-prefix trajectory is the structure gradient-based objectives are built for. We report a preliminary *supervised* fine-tuning study of whether training lifts a $\leq$8B agent inside that scaffold; reinforcement learning and on-policy distillation, the natural next stages, are left to future work. At matched sample size the supervised stage ties the untrained base rather than beating it — an honest first step that locates the accuracy lever in the on-policy methods left to future work.

5.1 What is trained, and how it is measured

We update only the weights of the reasoning policy — the language model that, each turn, emits a thought and a single Python action — fine-tuning it with LoRA adapters (Hu et al., 2021) on all linear projections, which keeps a run within a single 80 GB GPU. The perceptual backbone stays frozen: the vision-language model is reached only through an external HTTP endpoint that the policy calls via `batch_look`, and no gradient flows into it. Restricting training to the reasoning policy isolates the question of interest — whether weight-level training can lift the agent's accuracy — from any change in perceptual capability. The same model serves as both the policy under training and the policy under evaluation: trajectories are collected with the CodeAct scaffold, training updates

the policy, and evaluation runs the identical scaffold, so the policy that is trained is exactly the policy that is deployed.

The backbone is Qwen3.5-4B, a pivot from the Qwen3-8B model named in the original proposal. The 4B model is the one that holds the $\leq$8B leaderboard slot inside this agent scaffold, making it the relevant subject for a study aimed at the $\leq$8B tier; the larger, older-generation 8B model is not the incumbent, and using it would conflate a backbone change with the training intervention. Training uses verl (Sheng et al., 2024) with fully-sharded data-parallel (FSDP) model sharding.

Evaluation follows the §3.1 protocol — binary ANLS at threshold 0.9, native thinking disabled (the reasoner still writes its reasoning as free text around the code blocks, as explained in §3.1) — applied to the 4B agent with multiple independent samples per question to reduce per-rollout variance. Two scales are used: a small 13-question stratified mini-screen (covering all eight categories) at single-sample resolution for rapid checkpoint comparison during the hyperparameter sweep, and the full 80-question validation set, evaluated at the base's own n=8 sample size, to confirm the best configuration at matched power. A model and its baseline are compared per question on the same items, so improvements are assessed with paired statistics. Three reference points anchor the trained-model numbers: the untrained Qwen3.5-4B agent in the same CodeAct scaffold, the published $\leq$8B leaderboard entry at roughly 19 ANLS, and the 8B–35B-tier entry at roughly 37.5 (a different system from the like-valued Gemini 3 Pro frontier baseline).

5.2 Training signal

Rejection-sampling fine-tuning (SeqKD). No agent reference trajectories exist, so supervised targets are generated rather than annotated. The CodeAct scaffold with a 27B teacher rolls out many trajectories over the training pool (§5.3), and only those whose submitted answer is correct under the verifier are kept — rejection-sampling fine-tuning (Yuan et al., 2023) — after which the 4B policy is fit to the accepted trajectories with token-level cross-entropy. Because the teacher (27B) is larger than the student (4B), this is also a form of sequence-level knowledge distillation (Kim and Rush, 2016): hard-target imitation of filtered teacher trajectories, without an auxiliary distillation loss. (Unlike self-improvement variants of rejection sampling, the samples come from a separate, larger teacher rather than the policy itself.) This off-policy signal is the cheapest available and serves as a cold start: it places a frozen model onto the scaffold's turn structure and reduces the risk of a zero-reward-variance start in which no sampled rollout ever succeeds.

Beyond supervised imitation (future work). Supervised imitation can sharpen which answer the agent commits to but cannot raise its ceiling: at the 4B scale the agent already reaches a correct answer in some sample on far more questions than it lands consistently (pass@k much exceeds single-sample accuracy; §5.4). Converting that oracle headroom into reliable single-shot accuracy calls for an on-policy signal. The natural next stage is Group Relative Policy Optimization (Shao et al., 2024), which would sample a group of rollouts per prompt, score each with the answer-level ANLS verifier — augmented with shaping terms that penalize malformed or non-executing actions — and use the within-group reward spread as a baseline-free advantage. A per-token on-policy distillation signal — a KL to the teacher over the student's own rollouts — is a further option that combines imitation's density with reinforcement's distribution match (Song and Zheng, 2026). Both are on-policy, and both are left to future work for the reasons noted above.

5.3 The training-data pool

DocVQA-2026 provides no training split, so supervised fine-tuning requires a substitute corpus. We assemble a pool of short-document question answering from eight public DocVQA-family datasets, each row carrying a gold answer and a rule-based verifier. Because every prompt is independently scoreable, the pool supports rejection sampling directly: of the many teacher trajectories rolled out over it, we keep only those whose submitted answer scores correct, with no human annotation of agent trajectories required.

The eight sources span the document types the benchmark evaluates: scanned business documents (Mathew et al., 2020), high-resolution infographics (Mathew et al., 2021), charts (Masry et al., 2022), maps (Chang et al., 2022), multi-page documents (Tito et al., 2022), numeric financial reports (Zhu et al., 2022), slide decks (Tanaka et al., 2023), and long multi-page documents (Ma et al., 2024). They cover five of DocVQA-2026's eight categories — business report, infographics,

maps, science poster, and slide; comics, engineering drawings, and scientific papers have no suitable short-document source and are left to transfer.

We treat document length as a first-class axis of the mixture. Most sources are single-image, so the multi-page navigation that DocVQA-2026 demands — roughly 45% of its validation documents span multiple pages and a third exceed ten — would be almost absent from a naive pool. MP-DocVQA (filtered to at most three pages) and MMLongBench-Doc (median 28 pages, up to 80) supply that signal deliberately, so the pool exercises locating the right page, not only reading a single one.

The raw corpus is dominated by one source — MapQA alone contributes 483k of its 535k questions — so a per-source sampler draws a fixed quota from each dataset into a balanced pool of 11,464 prompts rather than pooling raw counts. The pool is screened for contamination against the DocVQA-2026 validation pages by perceptual-hash shortlisting followed by exact pixel comparison; the only reused images carry different questions in the two sets, so no validation answer is recoverable from the training data.

5.4 Preliminary results

We fine-tune the 4B agent on the leakage-free teacher trajectories and sweep LoRA rank and learning rate. Training is well behaved: every configuration converges smoothly (Figure 4), with final loss governed mainly by learning rate and only weakly by adapter rank — the expected behaviour for LoRA fine-tuning. Because a low training loss need not imply a better agent under the observation shift of a fresh multi-turn rollout, configurations are ranked by validation accuracy rather than by loss.

On the full validation set, evaluated at the base's own $n{=}8$ sample size for a matched comparison, the best configuration — LoRA rank 64, eight epochs — scores 23.0 ANLS against the untrained base's 22.34 (Table 5). The $+0.7$-point difference is small against the run-to-run spread of either model across the eight single-sample passes (± 3.44 for the base, ± 4.00 for the fine-tuned agent), so the two are tied, and the oracle and consistency measures agree: pass@8 is 53.8% against the base's 55.0% (tied), while self-consistency over the eight samples is in fact lower (17.5% versus 26.25%). An intermediate single-sample reading had suggested a larger lift (28.7 ANLS), but it rested on one favorable draw, and the eight-sample average regresses to the base. The honest verdict is a null: rejection-sampling fine-tuning on teacher trajectories does not move accuracy beyond the untrained base at the 4B scale.

Model	ANLS	pass@8	SC@8
Qwen3.5-4B base	22.34 ± 3.44	55.0	26.25
+ SFT (rank 64, 8 ep.)	23.0 ± 4.00	53.8	17.5

Table 5: Best supervised configuration on the full DocVQA-2026 validation set (80Q), at matched $n{=}8$. The ± figures are the population standard deviation across the eight single-sample passes; the $+0.7$-point ANLS difference is well within either spread (a tie), pass@8 is tied, and self-consistency (SC@8) is lower. Supervised fine-tuning does not beat the untrained base at matched power.

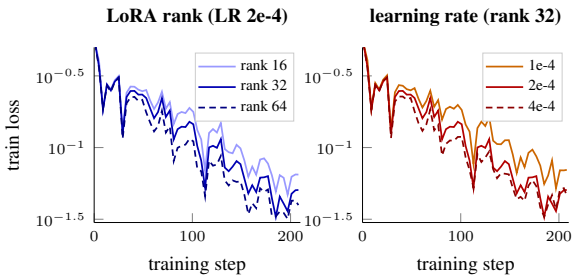

Figure 4: Supervised fine-tuning loss over the LoRA rank and learning-rate sweep (§5.4; representative learning rates shown), 208 steps. Every configuration converges smoothly; final loss is governed mainly by learning rate (right) and only weakly by adapter rank (left), consistent with learning rate being the dominant knob for LoRA fine-tuning. A low training loss need not translate into validation gains, so configurations are ranked by accuracy (Table 5).

This is consistent with the rest of the study. Supervised imitation of teacher trajectories is off-policy, and a fresh multi-turn rollout drifts from the demonstrated observations, so imitating trajectories does not transfer to the agent's own deployment distribution — even an in-domain set memorized to near-zero training loss only ties the untrained base. The remaining headroom is not extraction quality but completion: roughly 30% of rollouts exhaust their turn budget before submitting on hard multi-page documents, an efficiency failure that an on-policy reward, rather than off-policy imitation, is positioned to address. Supervised fine-tuning is therefore best read as a safe warm-start that places the model on the scaffold's turn structure, with the accuracy lever lying in the on-policy methods left to future work.

6 Related Work

Reinforcement learning for language models. Group-relative policy optimization (GRPO) replaces PPO's learned critic with a group-of-samples baseline, estimating advantages from the reward spread within a sampled group (Shao et al., 2024). A line of subsequent work refines its surrogate and normalization. Dr.GRPO removes length and standard-deviation normalizers that bias the gradient toward longer or lower-variance responses (Liu et al., 2025); DAPO contributes decoupled clipping, dynamic sampling, and token-level loss for stable long-CoT training (Yu et al., 2025); GSPO moves importance weighting from the token to the sequence level (Zheng et al., 2025); and CISPO clips importance-sampling weights rather than token updates to retain all-token signal (MiniMax et al., 2025). The reinforcement-learning stage left to future work (§5) would adopt GRPO with an ANLS verifier reward, and these variants delimit the design space for its surrogate.

Knowledge distillation. Sequence-level distillation (SeqKD) trains a student on teacher-generated outputs, an off-policy objective dense in tokens but disconnected from the student's own distribution. MiniLLM minimizes a reverse KL to avoid the mode-covering failure of forward KL on generative tasks (Gu et al., 2023), and generalized knowledge distillation (GKD) trains on student-sampled sequences to close the train–inference distribution gap (Agarwal et al., 2023). On-policy distillation extends this lineage to a dense per-token teacher signal computed over student rollouts (Song and Zheng, 2026). This SeqKD / on-policy-distillation contrast frames the signals explored in §5.

Reinforcement learning under agent-controlled context. Policy-gradient training assumes a single growing token sequence; harnesses in which the agent rewrites or compresses its own history break that assumption, and an emerging, not-yet-established line of work targets this regime directly. FoldAct shows that summary actions induce a policy-dependent, non-stationary observation distribution and repairs the gradient with purpose-built machinery (Shao et al., 2025); ReSum segments the trajectory at each compression and broadcasts trajectory-level advantage across segments (Wu et al., 2025); ContextCurator reports that an untrained context manager underperforms full-context prompting and overtakes it only after reinforcement learning (Li et al., 2026); and AdaCoM adapts the compression policy during agentic reasoning (Yi et al., 2026). Format-sensitivity of frozen models under prompt rewriting motivates the same concern (Sclar et al., 2023). §4 positions the append-only CodeAct scaffold against this frontier, choosing the prefix-preserving path for which mature training applies and leaving context-managing harnesses to future work.

Agentic document question answering. Recent systems pair a frozen vision–language model with an orchestrating agent. RVLM (Mayumu et al., 2026) drives perception through repeated on-demand calls to a frozen VLM perception tool — the same perception primitive used by the RLM harness we reuse (Zhang et al., 2025), a distinct system despite the one-letter-apart name. Multi-agent and tool-routing designs decompose document understanding across specialized roles (Borchmann et al., 2026; Han et al., 2025; Lassoued et al., 2026; Jin et al., 2025), while others add retrieval, visual search, or learned OCR control (Mohammadshirazi et al., 2025; Shen et al., 2026; Zheng et al., 2026; Wang et al., 2026). The scaffold here builds on the CodeAct paradigm of expressing agent actions as executable code (Wang et al., 2024) and on the ReAct interleaving of reasoning and action (Yao et al., 2022).

DocVQA-family benchmarks. The training-data pool of §5 and the evaluation of §2 draw on the document-VQA benchmark lineage: single-page DocVQA (Mathew et al., 2020), infographic and chart variants (Mathew et al., 2021; Masry et al., 2022), multi-page and multi-document settings (Tito et al., 2022; Van Landeghem et al., 2023), slide decks (Tanaka et al., 2023), and the long-document MMLongBench-Doc used for transfer trajectories (Ma et al., 2024). Scoring follows the ANLS metric introduced with ST-VQA (Biten et al., 2019).

7 Conclusion and Future Work

Across two model families and four reasoner scales, what governs document-agent accuracy on long, visually dense documents is not the size of the reasoner but whether it can actively control perception: an agent that writes code to crop, zoom, and query a frozen vision–language model on demand matches or exceeds much larger frozen mod-

els and far surpasses fixed-granularity tool agents. The advantage is mechanistically located — it concentrates where cropping recovers detail a whole-page read misses, collapses when visual perception is replaced by optical character recognition, and appears only once the base model is capable enough, in both reasoning and code, to drive the loop.

The two strongest scaffolds in this family tie at the 27B backbone, and the append-only member preserves the growing-prefix trajectory that established weight-level training assumes; it is therefore the principled target for training a small agent. Our preliminary supervised study at the $\leq$8B tier is an honest first step: at matched sample size, rejection-sampling fine-tuning on teacher trajectories ties the untrained base rather than beating it — off-policy imitation does not survive the observation shift of a fresh multi-turn rollout. Training a small agent to realize the active-perception advantage, with on-policy methods that match the deployment distribution, is the open direction we point to.

Limitations

The evaluation is confined to the DocVQA-2026 validation set (25 documents, 80 questions); the test set is held out by the competition portal, and the small validation size limits the resolution of per-category and per-slice estimates. The per-category and page-bucket analyses are computed on cross-model matched-configuration runs rather than the homogeneous baselines, so those figures are read as deltas, with the headline solver numbers (n=8) cited for absolutes. The CodeAct cells marked † (the 8B, 9B, and Gemma rows) are produced by an earlier implementation of that scaffold and are provisional pending a re-run, whereas the 4B and 27B CodeAct cells are the corrected, final n=8 loop; the active-perception conclusions rest on the RLM numbers regardless.

Generalization is demonstrated across two model families. The training study is deliberately preliminary and narrowed in scope: it covers only the supervised stage, which at matched sample size ties the untrained base rather than beating it, and the reinforcement learning and on-policy distillation set out in the proposal — the methods its null result points to — are left to future work. The account in §4 of why a context-managing scaffold has a higher but currently inaccessible ceiling is a hypothesis rather than a demonstrated result.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2023. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes.

Ali Furkan Biten, Ruben Tito, Andres Mafla, Lluis Gomez, Marçal Rusiñol, Ernest Valveny, C. V. Jawahar, and Dimosthenis Karatzas. 2019. Scene Text Visual Question Answering.

Łukasz Borchmann, Jordy Van Landeghem, Michał Turski, Shreyansh Padarha, Ryan Othniel Kearns, Adam Mahdi, Niels Rogge, Clémentine Fourrier, Siwei Han, Huaxiu Yao, Artemis Llabrés, Yiming Xu, Dimosthenis Karatzas, Hao Zhang, and Anupam Datta. 2026. Strategic Navigation or Stochastic Search? How Agents and Humans Reason Over Document Collections.

Shuaichen Chang, David Palzer, Jialin Li, Eric Fosler-Lussier, and Ningchuan Xiao. 2022. MapQA: A Dataset for Question Answering on Choropleth Maps.

Quentin Gallouédec and Kashif Rasul. 2025. Renderers: Token-in-Token-out Trajectory Rendering for Agentic RL. `https://github.com/PrimeIntellect-ai/renderers`. GitHub repository; prefix-invariant trajectory rendering write-up.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. MiniLLM: On-Policy Distillation of Large Language Models.

Siwei Han, Peng Xia, Ruiyi Zhang, Tong Sun, Yun Li, Hongtu Zhu, and Huaxiu Yao. 2025. MDocAgent: A Multi-Modal Multi-Agent Framework for Document Understanding.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models.

Yiqiao Jin, Rachneet Kaur, Zhen Zeng, Sumitra Ganesh, and Srijan Kumar. 2025. SlideAgent: Hierarchical Agentic Framework for Multi-Page Visual Document Understanding.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines.

Yoon Kim and Alexander M. Rush. 2016. Sequence-Level Knowledge Distillation.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention.

Aymen Lassoued, Mohamed Ali Souibgui, and Yousri Kessentini. 2026. ORCA: Orchestrated Reasoning with Collaborative Agents for Document Visual Question Answering.

Xiaozhe Li, Tianyi Lyu, Yizhao Yang, Liang Shan, Siyi Yang, Ligao Zhang, Zhuoyi Huang, Qingwen Liu, and Yang Li. 2026. Escaping the Context Bottleneck: Active Context Curation for LLM Agents via Reinforcement Learning.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. 2025. Understanding R1-Zero-Like Training: A Critical Perspective.

Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. 2024. MMLongBench-Doc: Benchmarking Long-context Document Understanding with Visualizations.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. ChartQA: A Benchmark for Question Answering about Charts with Visual and Logical Reasoning.

Minesh Mathew, Viraj Bagal, Rubèn Pérez Tito, Dimosthenis Karatzas, Ernest Valveny, and C. V. Jawahar. 2021. InfographicVQA.

Minesh Mathew, Dimosthenis Karatzas, and C. V. Jawahar. 2020. DocVQA: A Dataset for VQA on Document Images.

Nicanor Mayumu, Zeenath Khan, Melodena Stephens, Patrick Mukala, and Farhad Oroumchian. 2026. RVLM: Recursive Vision-Language Models with Adaptive Depth.

MiniMax, Aili Chen, Aonian Li, and 1 others. 2025. MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention.

Ahmad Mohammadshirazi, Pinaki Prasad Guha Neogi, Dheeraj Kulshrestha, and Rajiv Ramnath. 2025. ARIAL: An Agentic Framework for Document VQA with Precise Answer Localization.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting.

Jiaqi Shao, Yufeng Miao, Wei Zhang, and Bing Luo. 2025. FoldAct: Efficient and Stable Context Folding for Long-Horizon Search Agents.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.

Yucheng Shen, Jiulong Wu, Jizhou Huang, Dawei Yin, Lingyong Yan, and Min Cao. 2026. VISOR: Agentic Visual Retrieval-Augmented Generation via Iterative Search and Over-horizon Reasoning.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. HybridFlow: A Flexible and Efficient RLHF Framework.

Mingyang Song and Mao Zheng. 2026. A Survey of On-Policy Distillation for Large Language Models.

Ryota Tanaka, Kyosuke Nishida, Kosuke Nishida, Taku Hasegawa, Itsumi Saito, and Kuniko Saito. 2023. SlideVQA: A Dataset for Document Visual Question Answering on Multiple Images.

Rubèn Tito, Dimosthenis Karatzas, and Ernest Valveny. 2022. Hierarchical multimodal transformers for Multi-Page DocVQA.

Jordy Van Landeghem, Rubén Tito, Łukasz Borchmann, Michał Pietruszka, Paweł Józiak, Rafał Powalski, Dawid Jurkiewicz, Mickaël Coustaty, Bertrand Ackaert, Ernest Valveny, Matthew Blaschko, Sien Moens, and Tomasz Stanisławek. 2023. Document Understanding Dataset and Evaluation (DUDE).

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents.

Zhengren Wang, Dongsheng Ma, Huaping Zhong, Jiayu Li, Wentao Zhang, Bin Wang, and Conghui He. 2026. AgenticOCR: Parsing Only What You Need for Efficient Retrieval-Augmented Generation.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. 2025. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models.

Lu Yi, Runlin Lei, Liuyi Yao, Yuexiang Xie, Yuyang Li, Wenhao Zhang, Zhewei Wei, Yaliang Li, and Jian-Yun Nie. 2026. Learning Agent-Compatible Context Management for Long-Horizon Tasks.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, and 1 others. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. 2023. Scaling Relationship on Learning Mathematical Reasoning with Large Language Models.

Alex L. Zhang, Tim Kraska, and Omar Khattab. 2025. Recursive Language Models.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. 2025. Group Sequence Policy Optimization.

Yuanlei Zheng, Pei Fu, Hang Li, Ziyang Wang, Yuyi Zhang, Wenyu Ruan, Xiaojin Zhang, Zhongyu Wei, Zhenbo Luo, Jian Luan, Wei Chen, and Xiang Bai. 2026. Doc-V*: Coarse-to-Fine Interactive Visual Reasoning for Multi-Page Document VQA.

Fengbin Zhu, Wenqiang Lei, Fuli Feng, Chao Wang, Haozhou Zhang, and Tat-Seng Chua. 2022. Towards Complex Document Understanding By Discrete Reasoning.